GitHub project: https://github.com/naurjhanvi/Tales-OF-Hyderabad

Tales of Hyderabad Technical Design Document

Direct ordering, kitchen operations and central menu management | 9 November 2026

Tales of Hyderabad connects guest orders to outlet kitchens and a central menu workspace. The shipped slice covers guest ordering, cart and coupon checkout, simulated UPI/card records, kitchen status updates, menu and availability management, outlet offers, and order reporting. It uses a React web client, a Fastify API, and PostgreSQL. The payment provider, customer accounts, loyalty, catering and external messaging integrations are not connected.

The core design keeps prices and order state on the server. The browser submits selected menu options; the API validates availability, calculates the final amount, stores the order and publishes status events. The customer and kitchen screens receive changes through server-sent events, with periodic refresh as a recovery path.

Product slice

Area	Working flow	Key behavior
Customer ordering	Guest chooses an outlet, browses dishes, configures portions, spice and add-ons, applies a coupon, chooses UPI or card, and tracks the order.	Postal-code coverage and manual outlet choice; API-owned price, GST and discount calculation; private tracking token.
Kitchen display	Outlet staff sign in, review incoming tickets, move work through preparation, ready, rider handoff and delivery, and change item availability.	Outlet-scoped access, elapsed timers, late-order highlighting, immediate preparing status after acceptance, live event stream.
Menu admin	Central staff manage dishes, portions, add-ons, spice levels, active state and outlet availability.	One shared base menu for the three seeded outlets; sales are outlet-specific.
Offers and reporting	Admin creates coupons and item sales, then reviews order volume, discounts, revenue records, repeat customers, outlet activity and late orders.	Outlet-scoped coupons and sales; figures derive from stored order records.

Architecture and choices

Layer	Implementation	Reason
Web client	React with TanStack Start/Router, Vite and reusable components. Customer cart and private tracking tokens stay in browser storage; staff sessions use session storage.	Supports a responsive customer experience and separate customer, kitchen and admin entry points.
Application API	Fastify with TypeScript. Zod validates request bodies and query values. JWT claims carry staff role and outlet assignment.	Keeps checkout and permissions server-side and gives staff routes a clear role boundary.
Database	PostgreSQL stores menus, outlet coverage, item availability, sales, coupons, orders, payments and status history.	Relational constraints and transactions protect price snapshots, coupon redemption and order transitions.
Live updates	A committed status change sends PostgreSQL NOTIFY. Fastify streams snapshots and status events over server-sent events; the web client refreshes the affected order or outlet board.	Avoids a kitchen-wide refresh for each client action. A 30-second refresh remains as recovery if the stream disconnects.
Local runtime	Docker Compose runs PostgreSQL. Vite and the API run as separate Node services.	Keeps local setup reproducible while leaving production hosting choices open.

Request path: Browser to Fastify API to PostgreSQL

Data model

Record group	Stored information	Important rule
Outlets and coverage	Outlet address, hours, FSSAI display fields, prep and delivery estimates, and supported postal codes.	A customer can choose an outlet that serves the entered postal code; the customer can also choose manually.
Menu and options	Dish name, description, price in paise, diet mark, allergens, image key, portions, spice levels and add-ons.	Checkout re-reads active item and outlet availability before accepting an order.
Sales and coupons	Outlet-specific sale price and schedule; coupon type, value, limits, outlet scope, active dates and redemptions.	Sale and coupon values are recalculated by the API at checkout.
Orders and payment records	Order/customer snapshot, selected method, totals, applied discount, tracking-token hash, idempotency key and payment status.	Order lines keep price and item-name snapshots so later menu edits do not change prior orders.
Status history	Status, timestamp, optional note and staff actor for each change.	The API permits only the next valid kitchen transition; staff tokens are restricted to their outlet.

Main flows

Customer order

1. The customer enters a Hyderabad postal code or selects an outlet. The menu request is scoped to that outlet and hides unavailable items.
2. The browser sends item identifiers, quantities and option identifiers, plus customer delivery details, selected payment method and optional coupon code. It does not send prices.
3. The API validates the menu and outlet, calculates item prices, sale pricing, delivery fee, included 5% GST and coupon discount, then stores the order and a successful simulated payment record in one transaction.
4. The API returns an order code and a high-entropy tracking token. The browser stores the token and sends it in a private header for order reads and event streams; PostgreSQL stores only its hash.
5. The kitchen board accepts a new ticket and starts preparation in the same action. The API records both events and immediately sets the current customer status to preparing.

Kitchen updates

The kitchen board authenticates with a staff JWT and requests orders for its assigned outlet. A role check and an outlet check run before the API returns or changes records. Status changes use a database transaction; only after commit does the API publish a PostgreSQL notification for the customer and kitchen SSE streams. The browser reloads the corresponding order data when it receives a snapshot or status event. If a stream closes, the board continues periodic refresh and shows an update warning.

Menu and offer management

Central admins create or edit a menu item and its options, then set its availability by outlet. An item sale is attached to specific outlets and may have start and end times. Coupons may be global or restricted to outlets, with minimum order, discount cap, total use limit and per-phone redemption limit. The customer offer panel lists active outlet sales and eligible coupons; the cart validates the code again before checkout.

Automated preparation estimate

The estimate uses completed orders from the same outlet during the previous 30 days. It measures elapsed time from order placement until the first ready event. With at least five completed orders, the API uses the median. With fewer examples, missing history or an invalid estimate, it falls back to the outlet minimum. It adds three minutes per active kitchen order, capped at 15 minutes, and two minutes per additional basket item, capped at 10 minutes. The final promise is never below the outlet minimum.

Situation	Estimate basis	Safeguard
Fewer than five recent completions	Outlet minimum: 20, 22 or 25 minutes in the seed.	Avoids treating sparse data as reliable.
At least five recent completions	Median placement-to-ready time for the outlet.	Uses local operating history instead of a chain-wide average.
Current kitchen load or larger basket	Bounded queue and basket buffers.	Limits the extra time and preserves the minimum floor.

This is a transparent statistical heuristic, not a trained machine-learning model. The outlet minimum provides the conservative fallback. A manager override for an individual estimate is not implemented; adding an override and a reviewable estimate history is a next step.

Design decisions

Screen group	Design choice
Customer entry and menu	Warm ivory background, restrained chilli and saffron accents, food photography, clear diet/allergen marks, scannable prices and mobile-first spacing.
Cart and checkout	Visible item options, coupon entry and savings, order total and delivery details in one short path. GST inclusion is stated; no payment credentials are requested.
Order tracking	A compact status timeline emphasizes the active kitchen stage, expected ready time and order contents.
Kitchen	Dark, high-contrast board with large ticket details, timers and color marking for late work; controls support a touch tablet.
Admin	White, translucent glass surfaces with dark readable type, consistent navigation, outlet/date filters, actionable reports, menu controls and offer management.

Security and data handling

- Passwords are stored as bcrypt hashes. Staff API routes require a signed JWT; role checks separate menu administration from kitchen actions, and kitchen tokens are outlet-scoped.
- Zod schemas validate API inputs. Checkout uses server-side prices, idempotency keys, a database transaction and a private order tracking token whose hash is stored in the database.
- Environment examples contain blank secret fields. Local credentials and signing keys belong in ignored environment files, not source control. The ZIP contains documentation only, not local environment files or database exports.
- Guest checkout stores delivery contact details with the order. Marketing messages, customer accounts, consent management and a retention/deletion workflow are not implemented; a production deployment needs a reviewed privacy and retention process.

Assumptions and limits

- Assumption - the three seeded outlets, menu items, prices, hours, postal-code coverage and FSSAI display values are illustrative. The FSSAI numbers are not verified licences. The client brief describes 14 outlets and three cloud kitchens; this build models only Kondapur, Gachibowli and Kukatpally.
- Assumption - the seeded customer names, addresses and phone values are fictional. The starter orders are for review and do not represent real customer records.
- Assumption - all displayed menu prices include 5% GST. Delivery fee and menu values are starter values for this product slice, not client-approved commercial prices.
- Assumption - postal-code coverage is used as the delivery-area rule. GPS distance, geocoding, rider allocation and delivery-partner integration are not implemented.
- UPI and card are simulated. The selected method is recorded, but no payment gateway, settlement, refund or charge is performed.
- The shared menu uses one base price across the three seeded outlets. City-specific price books, time-based menu switching, loyalty points and customer accounts are not implemented.
- Catering intake and manager confirmation, WhatsApp/SMS updates, explicit late-order manager notifications, manager ETA overrides and reorder suggestions are not implemented.

Verification and next work

Backend unit tests cover the estimate fallback, recent-history calculation, load and basket buffers, percentage and fixed discounts, discount caps and the subtotal ceiling. The frontend test checks application route registration. The backend TypeScript build, frontend TypeScript check, lint command and production frontend build were also run. The local seed completed successfully and created the three coupons and three starter orders. Automated end-to-end tests for browser checkout, staff authorization and SSE reconnection are not included.

Next work: connect a payment provider; add loyalty and customer consent; implement catering intake and confirmation; add outlet price books and time-based menus; let managers override estimates; send and audit late-order alerts; expand seed coverage to all 17 client locations; and run integration and browser end-to-end tests before a hosted release.